

Basic Data Types in Java

This document provides an overview of the basic data types in Java, including their characteristics, sizes, and examples. Understanding these data types is crucial for effective programming in Java, as they form the foundation for data manipulation and storage in applications.

1 Primitive Data Types

J
k
c

1
#

#

#

#####

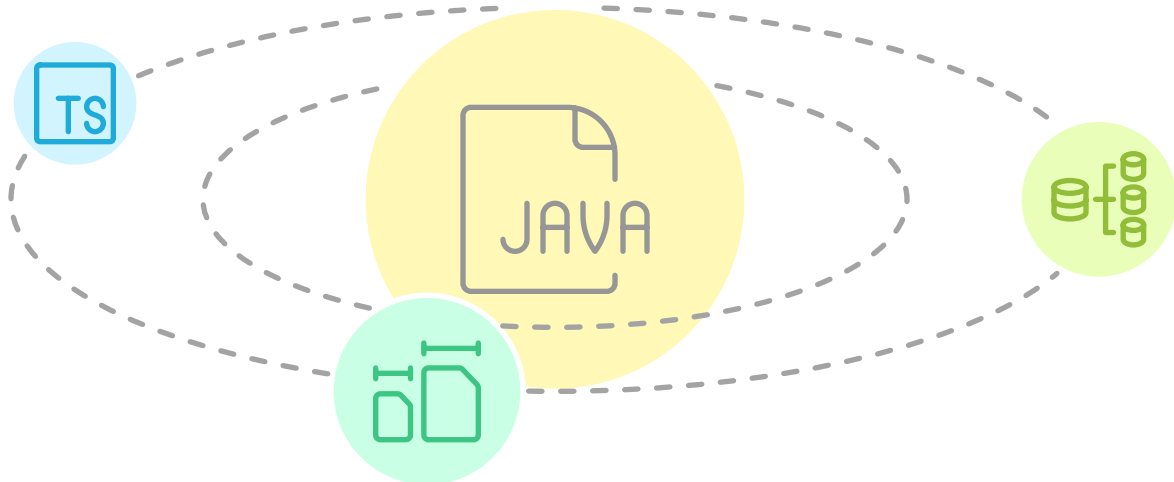
#####

#####

#####

#####

Java Data Types Overview



Characteristics

Describes the attributes of each data type

Sizes

Specifies the memory allocation for each data type

Examples

Provides practical instances of data type usage

Made with Napkin

- **Size:** 32 bits
- **Range:** -2,147,483,648 to 2,147,483,647
- **Example:**

```
int i = 100000;
System.out.println("Integer value: " + i);
```

- **Size:** 64 bits
- **Range:** -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
- **Example:**

```
long l = 100000L;
System.out.println("Long value: " + l);
```

1.2 Floating-Point Types

- **Size:** 32 bits
- **Range:** Approximately $\pm 3.40282347E+38$ [6-7 significant decimal digits]
- **Example:**

```
float f = 10.5f;
System.out.println("Float value: " + f);
```

- **Size:** 64 bits
- **Range:** Approximately $\pm 1.79769313486231570E+308$ [15 significant decimal digits]
- **Example:**

```
double d = 20.5;
System.out.println("Double value: " + d);
```

1.3 Character Type

a. char

- **Size:** 16 bits
- **Range:** 0 to 65,535 [Unicode characters]
- **Example:**

```
char c = 'A';  
System.out.println("Character value: " + c);
```

1.4 Boolean Type

a. boolean

- **Size:** Not precisely defined (but typically 1 bit)
- **Values:** true or false
- **Example:**

```
boolean isJavaFun = true;  
System.out.println("Is Java fun? " + isJavaFun);
```

2. Wrapper Classes

Java provides wrapper classes for each of the primitive data types, which allows them to be treated as objects. This is particularly useful when working with collections.

Examples of Wrapper Classes:

- Byte, Short, Integer, Long, Float, Double, Character, Boolean

Example:

```
Integer intObject = Integer.valueOf(i);  
System.out.println("Integer object: " + intObject);
```

3. Type Conversion

Java supports both implicit and explicit type conversion.

Implicit Conversion (Widening)

- Automatically converts a smaller type to a larger type.
- Example:

```
int num = 100;  
long longNum = num; // Implicit conversion  
System.out.println("Long number: " + longNum);
```

Explicit Conversion (Narrowing)

- Requires casting to convert a larger type to a smaller type.
- Example:

```
double dNum = 100.04;  
int intNum = (int) dNum; // Explicit conversion  
System.out.println("Integer number: " + intNum);
```

Conclusion

Understanding the basic data types in Java is essential for any Java programmer. This document has outlined the primitive data types, their characteristics, and provided examples to illustrate their usage. Mastery of these data types will enhance your ability to write efficient and effective Java programs.